

README / Setup Guide

Overview

This application is an employee leave management system designed to streamline the process of submitting, approving, and tracking leave requests within an organization. It provides a user-friendly interface for employees to submit leave requests and for HR personnel to approve them. The system ensures that leave requests are processed efficiently and that leave balances are accurately tracked.

Key Features

- **Employee Leave Request Submission:** Employees can submit leave requests through a user-friendly interface, specifying the leave type, start date, and end date.
- **Leave Request Approval Workflow:** HR personnel can approve or reject leave requests, updating the status of each request.
- **Leave Balance Tracking:** The system calculates and displays the leave balance for each employee, ensuring that leave requests are within the allowed limits.

Prerequisites

- **Java 11+:** Ensure you have Java 11 or higher installed.
- **PostgreSQL:** A PostgreSQL database server is required.
- **Maven:** Maven 3.6+ for building and managing dependencies.
- **Node.js 16+:** For frontend development if applicable.
- **Git:** For cloning the repository.

Project Structure

The repository is structured as follows:

- **backend/:** Contains the Spring Boot backend responsible for handling business logic, API endpoints, and database interactions.
- **frontend/:** Contains the frontend code if applicable.
- **shared/:** Contains shared utilities if any.
- **docs/:** Contains documentation and API specifications.

Installation & Setup

Backend Setup

1. Clone the repository: `git clone <url>`

2. Navigate to the project: `cd project-name`
3. Create virtual environment: `mvn clean install`
4. Set environment variables: `export DB_URL=jdbc:postgresql://localhost:5432/leavedb` and `export DB_USER=yourusername` and `export DB_PASS=yourpassword`
5. Start server: `mvn spring-boot:run`
6. Verify: Backend is running on `http://localhost:8080`

Frontend Setup

1. Navigate to frontend: `cd frontend`
2. Install dependencies: `npm install`
3. Build/prepare assets: `npm run build`

Configuration

Required Environment Variables

- **DB_URL**: Controls the PostgreSQL database URL, format: `jdbc:postgresql://host:port/dbname`, example value: `jdbc:postgresql://localhost:5432/leavedb`
- **DB_USER**: Controls the PostgreSQL database username, format: `string`, example value: `yourusername`
- **DB_PASS**: Controls the PostgreSQL database password, format: `string`, example value: `yourpassword`

Optional Environment Variables

- **SERVER_PORT** (default: 8080): Controls the port on which the server runs, format: `integer`, effect if changed: Server runs on the specified port.

Running the Application

Starting the Backend

1. Ensure virtual environment is activated
2. Set environment variables: `export DB_URL=jdbc:postgresql://localhost:5432/leavedb` and `export DB_USER=yourusername` and `export DB_PASS=yourpassword`
3. Start server: `mvn spring-boot:run`
4. Verify: Backend is running on `http://localhost:8080`

Starting the Frontend

1. Navigate to frontend directory: `cd frontend`
2. Start dev server: `npm run dev`
3. Open browser: `http://localhost:3000` (or displayed URL)
4. Verify: UI loads and connects to backend

Verification Checklist

- [] Backend health check responds at GET `/api/health` with 200 OK
- [] Frontend loads without console errors
- [] Basic workflow completes end-to-end (e.g., Employee submits a leave request and HR approves it)
- [] API endpoints respond with expected data format

Troubleshooting

Backend Issues

- **Port already in use:** Change `SERVER_PORT` env var or kill process on port 8080
- **Module not found:** Run `mvn clean install` again
- **Database connection error:** Verify connection string in config, check service is running
- **Authentication error:** Verify API keys/tokens in `.env` are correct and not expired

Frontend Issues

- **Blank page or won't load:** Check browser console for errors, ensure backend is running
- **API connection error:** Verify backend URL in config, check CORS settings
- **Module resolution error:** Delete `node_modules` and run `npm install` again
- **Build fails:** Ensure Node.js version matches requirements, clear cache: `npm cache clean --force`

Common Workflows

Submit a Leave Request

1. Navigate to the leave request submission page.
2. Fill in the leave request form with employee ID, leave type, start date, and end date.
3. Click the submit button.
4. Verify: Leave request is submitted successfully and a confirmation message is displayed.

Approve a Leave Request

1. Navigate to the leave request approval page.
2. Select a leave request to approve.
3. Click the approve button.
4. Verify: Leave request status is updated to "APPROVED" and a confirmation message is displayed.

Deployment Notes

- Ensure database credentials are securely managed.
- Configure environment variables for production settings.
- Set up a reverse proxy for secure API access.

Getting Help

- Check logs: Backend logs in console, frontend logs in browser DevTools
- Review error message and stack trace for clues
- Check environment variables are set correctly
- Refer to the API Documentation and User Guide for more details